

BEFORE

OLD User Class

```
public class User {  
    // GRASP: Information Expert - User handles its own data  
    @Id  
    @GeneratedValue(strategy = GenerationType.IDENTITY)  
    private Long id;  
  
    private String username;  
  
    private String password;  
  
    private String email;  
  
}
```

AFTER

Now uses Builder as a Creational Pattern and added a method that makes it easier to update the details of the entity

```
@Entity
@Getter
@Builder // Creational Pattern: Builder
@AllArgsConstructor
@NoArgsConstructor
@Table(name = "users")
public class User{

    // SOLID: SRP - Represents and manages user domain state only

    // GRASP: Information Expert - User handles its own data

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String username;

    private String password;

    private String email;

    // OOP: Encapsulation
    public void updateDetails(UserRequest req) { 1 usage
        this.username = req.username();
        this.password = req.password();
        this.email = req.email();
    }
}
```

BEFORE

In UserServiceTest, this is what I did to mocking and manually setting the username email and id

```
// Arrange
Long id = 1L;

User existingUser = new User();
existingUser.setId(id);
existingUser.setUsername("old");
existingUser.setEmail("old@mail.com");

UserRequest req = new UserRequest( username: "SigmaUpdated", password: "newpass", email: "sigma@updated.com");

User savedUser = new User();
savedUser.setId(id);
savedUser.setUsername("SigmaUpdated");
savedUser.setEmail("sigma@updated.com");
```

AFTER

Using builder, this is now the pattern for creating a user

```
UserRequest req = new UserRequest( username: "sigma", password: "pass", email: "s@mail.com");

User user = User.builder()
    .username("sigma")
    .password("pass")
    .email("s@mail.com")
    .build();

User saved = User.builder()
    .id(1L)
    .username("sigma")
    .email("s@mail.com")
    .build();
```